AI Storyteller Application

Project Documentation

Jade Pitkänen

jade.pitkanen18@gmail.com

Latest Update: 06/05/2026

Table of Contents

1 Documentation Overview

This document includes information on 'AI Storyteller' application which is publicly available on GitHub (https://github.com/Jaduli/AI-Storyteller). The application uses an external AI service to continue a user-generated story based on five context features: recent story, story summary, memories, and user-generated story essentials and context cards. The result is an application that can be used as a never-ending, customizable storyteller which can be used for entertainment or as a writing assistant.

Inspiration for this project has been gained from similar AI storytelling programs such as AI Dungeon (https://aidungeon.com/) and Novel AI (https://novelai.net/). Compared to them, this application has the benefit of running locally, allowing more detailed control over story generation at a cheaper cost by using a local AI and/or low-cost external APIs. Latency can also be much lower, and the model and API provider can be chosen freely from APIs that follow OpenAPI specifications.

The application has been made with a focus on secure programming practices (chapter 4). It includes, for example, safe file handling, error handling, and protection against XXS attacks, SQL injections, directory traversal, and CORS issues.

2 Structure of the Application

The application has been containerized with Docker. There are two versions of the app: one without local AI, and one with a local AI model run with Ollama (GPU mode, default model: llama3.1:8b). Running the local AI model requires an NVIDIA GPU with compatible drivers and at least 8 GB of VRAM. Running the app without local AI works on any hardware. In total, the app has three Docker images: frontend, backend, and Ollama (local AI, only used on GPU mode).

2.1 Technologies Used

The following table contains the technologies used in the making of this application and explanations for their use.

Table 1: Technologies used in the creation of the application.

Frontend	
Technology	**Purpose**
Vue.js	Vue.js is used to create the frontend application. I chose Vue due to being familiar with it through other projects and because it contains XXS protection by default along with many useful, pre-made components.
Nginx	Nginx is used as a reverse proxy between frontend and backend. This prevents CORS issues as frontend and backend are served from the same origin. It also improves security by not exposing the backend directly to frontend users. Nginx could be expanded to be used as a load balancer and an IP blocker if support for multiple users was added to the application.
Backend	
Python	The backend is built mainly on Python as I have experience using it through other projects. Python is used as the main backend coding language which handles routes, save files, and the database.
Flask	Flask is a light-weight framework for Python which has been used for routing. I chose Flask over Django as the current app does not require complex features such as user management, but future development could switch to using Django.
SQLite	The app contains a SQLite database for storing memories. I chose SQLite due to its light weight and due to being familiar with it through other projects.

FAISS	FAISS (Facebook AI similarity search, https://faiss.ai/index.html) is implemented to work alongside the database. FAISS uses a transformer to embed text, allowing the comparison of recent story content to memories stored in the database. The most relevant memories can then be used as context in story generation, improving story consistency and memory recall. Without FAISS, memory fetching would be limited to randomized or most recent memories.
Other Technologies	
Docker	Docker is used to containerize the application, allowing each component to run efficiently on different systems.
Ollama	Ollama (https://ollama.com/) is used to run a local AI model that can be used for memory and summary generation. I chose Ollama due to being familiar with it through other projects.

2.2 How to run the Application

Docker is required to run the app: https://www.docker.com/products/docker-desktop/.

With Docker running, navigate to "AI-Storyteller" folder in your terminal and run the command ‘docker compose up --build’ to build and run the app without local AI (=> all AI use is done with an external API).

Once running, frontend can be accessed at http://localhost:5173/.

If you have an NVIDIA GPU with at least 8 GB of VRAM and compatible drivers, you can run the app with a local AI for summary and memory actions (GPU mode). To use local AI, build and run the app with the command ‘docker compose -f docker-compose.yml -f docker-compose.gpu.yml up --build’. Running the AI locally will reduce cloud API rate limits and cost. Local story continuation is not supported because competent storytelling should be done with a large (20B+ parameter) model, which would require a large amount of memory to use. Smaller models struggle with consistency and creativity and have limited use cases.

The full container takes about 25-35 GB of space to run, mainly due to Docker images and the local AI model (only installed on GPU mode). First time set up may take 20+ minutes to build depending on selected mode and internet connection speed. After startup, the frontend page will load fully once backend is ready to accept API calls.

Add your cloud AI API key and URL in a .env file in the project root folder. The API key is used only for API calls to your selected cloud provider. You can also add default models for external API calls. The API key needs to be added to the root folder before building the application.

Example AI, GroqCloud: https://console.groq.com/home
API URL: https://api.groq.com/openai/v1/chat/completions

Groq provides a free model: llama-3.1-8b-instant (rate limits apply, use less context, recommended only for testing purposes).

Best quality/price AI, DeepSeek: https://platform.deepseek.com/
API URL: https://api.deepseek.com/chat/completions
Model name & Pricing can be found here:
https://api-docs.deepseek.com/quick_start/pricing

The container can be stopped with CTRL+C in the terminal. Once built, the container can also be run directly through the Docker Desktop app.

2.3 How to use the Application

Once you have the application running, begin a story by writing in the editor or load a previous story with its filename. Click on 'Continue Story' for the AI to continue the story from where it left off. The AI uses all of summary and story essentials, up to five relevant context cards, and up to two most recent + two most relevant memories for context. The limit of recent story used as context can be adjusted in the 'Settings' menu. Note that the limit (1 token = 4 characters) is only an approximation and that the limit has no effect on other context fields. You may enable a setting to display total tokens used in each API call. The full context & system instructions used in story generation can be seen in the 'Sent Context' tab.

The application includes a simple example scenario which can be accessed with the filename 'example' or 'example.json'. After a successful story continuation, you can look through the 'Sent Context' tab to see how different context fields are included in story generation.

Additional story generation instructions, such as storytelling style or content restrictions, can be written in the Instructions tab. Default prompts for story, memory, and summary generation can be found in backend/default_prompts.py file.

The story is saved automatically after every Continue action. It can also be saved manually with the Save button. One backup save is created for each file which can be loaded with the filename {filename}_backup.json. Saved files and their backups can be found in backend/files folder.

Past memories for each story are saved in a database which can be found in backend/data/memory.db file. Memories can be viewed and edited with an SQL compatible database editor, e.g DBeaver (https://dbeaver.io/). As summaries and memories are generated with AI, hallucination and metatext may be included in the output. For this reason, manual context editing may be required to lower token usage and to keep the story consistent.

2.4 Application Components

Most context components of the application are similar to the ones found on AI Dungeon (https://play.aidungeon.com/). The main components are:

- Recent story: This is the part of the story that the AI will continue directly from. Recent story can be viewed and edited in the 'Story Editor' tab.
- Instructions: This tab can be used to add storytelling instructions for story generation. Instructions can include, for example, storytelling style (e.g. point of view, book/world references), content restrictions (e.g. age restrictions, no mature/distressing content), and story genre (fantasy/historical/horror etc.).
- Story Essentials: This tab can be used to include important facts of the story's world, plot, or characters, e.g. protagonist details (name, looks, personality) and world details (city, universe, time, etc.). Story essentials will always be included in story context so keeping it as short as possible will help with reducing token usage.
- Context Cards: These work similarly to story essentials, but a context card will trigger only if at least one of the set keywords is found in recent story. Context cards can be used to, for example, add details about side characters, locations, or past events. If certain context is relevant to the story only at certain times, e.g. when a side character is being interacted with, it should be added as a context card rather than put into story essentials.
- Summary: The story summary is generated automatically as the story progresses. It can be viewed and edited in the 'Summary' tab. The summary generally includes the most recent past content in order to keep story direction consistent.
- Memories: Story memories are generated automatically. Only context older than the recent content window is memorized. Currently, if memories have been created, two most recent memories and up to two most relevant memories are used in story generation. Memories allow story generation to recall events that have happened in the past, allowing better consistency and more meaningful storytelling.

Storytelling can also be influenced by generation settings. The Settings menu includes the following components:

- Main Model Name: The main model that will be used to continue the story. Model names can be set in the .env file to keep them consistent after restarting the app.
- Memorize/Summarize Model Name: The model is used for summary and memory actions. The model can be set separately because memory and summary generation do not require a large model to run, allowing a cheaper and more efficient model to be used.
- Recent Story Token Limit: This allows limiting token usage by using only a set length of recent story as context. Length calculation is done by approximating each token to be 4 characters long, which may wary based on content and language. Currently the maximum limit is 32000 tokens, but the best consistency/cost ratio is achieved with 4000-6000 tokens. The limit does not affect the length of other components used in story generation. Memories and summary add 1000-2000 tokens to the context in total, and other components (instructions, essentials, context cards) add tokens based on content written by the user.
- Top P and Temperature: These control randomness in the AI output. Higher values mean more randomness and creativity, lower values improve consistency with story context (e.g. story essentials and memories). Temperature is limited to values between 0–2 and Top P to values between 0–1.
- Maximum Output Tokens: This controls the number of tokens returned by each continue action. In most APIs, output tokens cost more to use than input tokens. The default value is set to 200, but the value can be set between 10–1000. Note that the AI may sometimes return less content than the token limit allows.
- Show API Call Token Usage: This setting can be enabled to show total tokes used in each API call, including summary and memory creation.
- Use Local AI: This setting is only visible if the application is built on GPU mode. If checked, memory and summary actions will be run with local AI (requires compatible GPU). If unchecked, memories and summary will be created with an external AI model set by the user.

3 Code Structure

The application mostly contains code made with Vue.js (frontend) and Python (backend). Backend, frontend, and local AI components can be found in their respective folders.

3.1 Frontend

The main page of the application can be found in src/App.vue file, which uses the components found in the components folder. Most story context and editing features are found in StoryEditor.vue, settings in SettingsMenu.vue, and context card features in ContextCards.vue and ContextCard.vue. Nginx configuration can be found in nginx.conf. Some styling and colors have been used from the Vue template, which can be found in scr/style.css. Default application settings and styling can be found in App.vue.

The frontend code uses snake case for constants and variables, and camel case for functions in order to differentiate easily between the two. This is only a personal preference as the standard practice in Vue would be to use camel case for all.

The frontend application can be accessed at http://localhost:5173/.

3.2 Backend

The main routes used in backend can be found in the app.py file, and the required imports in requirements.txt. The utils.py file contains some utility functions used in app.py. The database.py file contains functions for creating and managing the SQLite database and FAISS features. Default prompts for story, memory, and summary generation can be found in default_prompts.py.

Backend runs on port 5000.

3.3 Local AI

The ai folder contains a Dockerfile and a start.sh script. The script starts the Ollama service and installs the model used by the application (llama3.1:8b). The model is stored in a Docker volume called ollama_data.

Local AI runs internally at http://ai:11434/. The running status of the service can be checked at http://localhost:11434/, where the page will say ‘Ollama is running’ if the service is ready.

4 Secure Programming Features

The application has been made with a focus on secure programming practices. The table below shows which threats have been taken into account while creating the application.

Table 2: Relevant threats that have been addressed while making the application.

Threat	Addressed At	Addressed By
XSS attacks	Frontend	Vue contains XSS attack prevention by default. This is done by escaping user input, allowing safe handling of text. For example, HTML code is rendered harmless, and no v-html has been used in the code to bypass this.
CORS issues	Frontend nginx.conf	The application uses Nginx which acts as a reverse proxy and serves frontend and backend from the same origin, preventing CORS issues.
SQL injection	Backend database.py	Inserting data into the database is done by passing parameters, preventing the insertion of malicious code as all special characters are rendered harmless.
Directory traversal	Backend app.py	Directory traversal is prevented when saving or loading files by ensuring the used path is correct (backend/files/).
Malicious file uploads	Front + backend StoryEditor.vue app.py	Filenames are ensured to end with .json, preventing the upload of files in the wrong format. Filenames are also limited to contain only letters, numbers, dashes, and underlines. File size is limited to 5 MB to prevent maliciously large files.
File/save corruption	Backend app.py	One backup is made for each save as a prevention measure against losing data due to unexpected errors or corruption.
Network/API call errors	Front + backend App.vue utils.py	Error handling has been done with error codes and messages. For example, the application will load only once a connection to the backend can be made, and external API call errors (e.g. API key unauthorized, model not found) are displayed correctly. External API calls are also given a 30 second timeout to prevent infinite calls.

Invalid values	Front + backend	Invalid values have been addressed with constraints and fallback values. For example, SettingsMenu.vue file prevents the insertion of values beyond the limits with fallbacks and clamping. Values are also checked to prevent errors such as creating a save without a filename.
Unexpected errors	Front + backend logging & status_message in StoryEditor.vue	Error handling has been done in case of unexpected errors. Errors will be displayed as a status message in frontend, and API call messages are automatically logged in the terminal.

5 Application Testing

Testing has been done manually through using the application and inputting invalid values to see how the application behaves. Some bugs were found in this way, and fixes were made to prevent them. Beyond GitHub Copilot coding assistant (chapter 7.2), no special testing programs or pipelines were used while creating the app.

The following table shows some bugs that were found in testing and the actions that were taken to fix them.

Table 3: Bugs found during testing and actions done to fix them.

Bug / Issue	Files / Location	Fix
Pressing the 'Continue' multiple times caused multiple API calls to be made simultaneously.	StoryEditor.vue	Added active_requests counter which tracks if any API call is in progress and disables save, load, set limit, and continue buttons until previous requests have completed.
Settings menu input fields allowed the insertion of invalid values (too high/low, letters when a number is expected).	SettingsMenu.vue	Added fallback values and clamping.
Changing recent story token limit corrupted story content.	SettingsMenu.vue StoryEditor.vue	The bug happened due to the editor displaying only recent content rather than the full content, causing some content to be lost or repeated when the token limit was changed. Fix was done by updating full content with the old token limit, and only then updating the editor to display recent content with the new limit (context_length watch function in StoryEditor.vue).
Loading a JSON save file without an ID object caused bugs with the database that uses IDs.	backend/app.py	Added requirement for save files to have a valid story ID when loading or saving files.

There was only one FAISS index, causing errors when searching for relevant memories.	backend/data database.py	Added a FAISS index for individual stories based on story ID.
Frontend loaded before backend was ready, causing issues with config and API calls.	App.py	LoadConfig function in App.py was added to attempt to connect to backend every 2 seconds, then load the app fully once a connection is successfully made.
Errors occurred when trying to run local AI without a compatible GPU.	Docker compose	Added a docker compose file which builds the app without local AI, allowing the app to be used on any machine. This also saves resources as the local AI model requires space to install.
Memory and summary cursors would move even when errors occurred during summary or memory creation.	StoryEditor.vue	Changed extractPastContent function to return cutoff index and moved the cursor value updates to happen only after API calls have successfully executed.

6 Future Improvements

The application currently runs only locally but could be expanded into a fully working web service. The service would require features such as user management and a wallet for AI usage. Other features that could be added to the application are the following:

- A page to view, load, and delete save files.
- A page to view and edit database memories.
- A page where status messages and token usage are logged to. Currently some messages are lost if multiple API calls happen concurrently (e.g. summary creation can override the token usage message after a continue action).
- Content generation restrictions. Currently, there are no content generation restrictions beyond the model's and API's own guardrails. New modes could be introduced which limit the content the AI can generate. A second layer of AI could also be added which reads the generated output and checks if it follows the set content rules.
- Saving story content in a database instead of a JSON object. Currently, if the story's content becomes long, loading and saving the full content every time consumes unnecessary resources. The story could instead be saved in batches inside a database and only the required story content be exchanged between front and backend when saving or loading files.
- Story components that haven't been edited between API calls could be cached in backend, which would save resources.
- The ability to have summary and memory generation run through a separate API provider could be added as not all providers (e.g. DeepSeek) have smaller models available.
- The app could include more RPG game-like features, such as a player card which includes player condition, inventory, relationships, and other factors that can affect story generation.
- An ability to install/load different local models could be added. The current llama model was chosen because it runs well on my PC, but other computers could run larger models or require a smaller one.

7 Notices

The following notices regard the use and creation of the application.

7.1 Usage & Content Safety Notice

You are free to use and edit the program for personal use. Commercial use is not allowed. You are personally responsible for any content you create with the application.

By default, there are no restrictions set in place for content generation. Most API providers and models include some guardrails against dangerous or illegal content, but the extent of the restrictions may vary. The AI can generate content that could be considered dangerous, pornographic, or disturbing, especially if prompted to do so. For this reason, the application should never be used by minors without adult supervision.

7.2 AI Usage in the Creation of the Application

This application has been made in VSCode with GitHub Copilot (free version for TUNI students) turned on. Model used was GPT-5 mini. Copilot has helped with commenting, debugging, autocompletion, CSS, and generating useful functions such as for trimming text. AI has NOT been used in the overall structure, project idea, or technology selection. I have made sure to implement every feature in a way that I personally understand the code and its function even in the parts where AI has been used.

Default prompts (in backend/default_prompts.py) have been improved with the help of ChatGPT-5. As FAISS is a new technology for me, ChatGPT-5 has also been used to help generate and understand code found in backend/database.py.